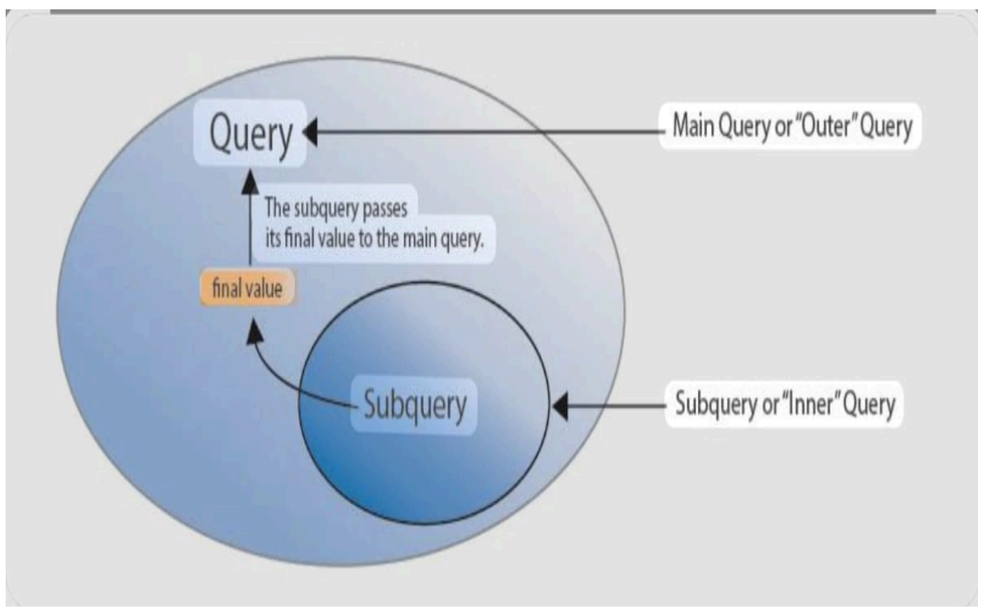
STYLECART-FASHION E-COMMERCE DATABASE MANAGEMENT SYSTEM

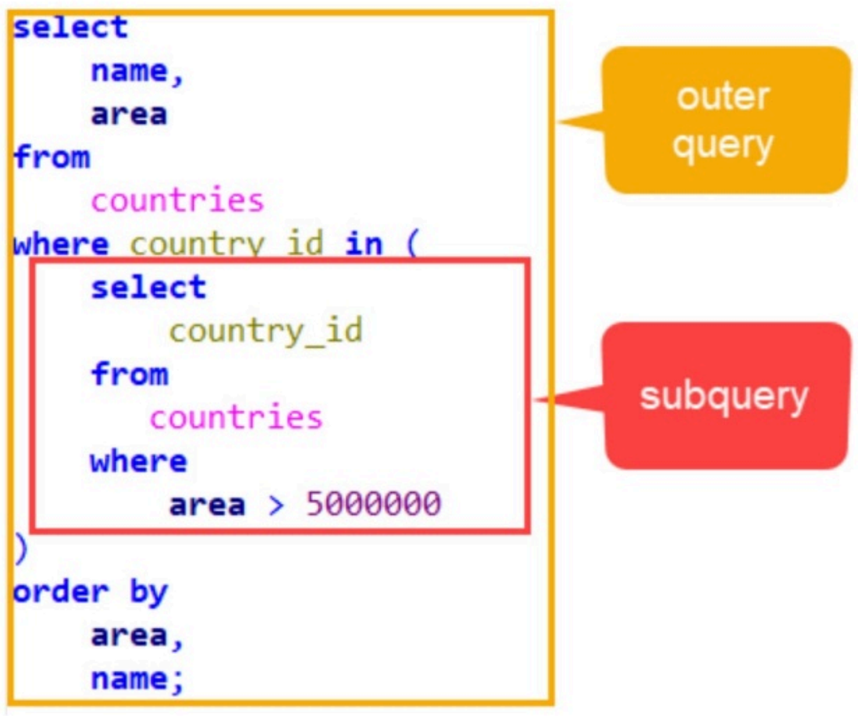
DBMS – ADVANCED SQL QUERY SYSTEM

1.subqueries and nested queries

Definition: A subquery is a query written inside another SQL query. It is used to retrieve data based on the result of another query.



basic structure



types of subqueries

Single Row Subquery – Returns only one row.

Multiple Row Subquery – Returns more than one row.

Multiple Column Subquery – Returns more than one column.

Nested Subquery – A subquery written inside another subquery.

Correlated Subquery – Depends on the outer query.

2.find product above average price

Purpose: To display products whose price is greater than the average price of all products.

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price > (SELECT AVG(Price) FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       101
Laptop
     50000

       102
Mobile
     20000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

       107
Tablet
     15000
```

2.1product below average price

Purpose: To display products whose price is less than the average price.

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price < (SELECT AVG(Price) FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
       103
Shirt
      1000

       104
Chair
      5000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------

       105
Book
       500

       106
Headphone

PRODUCT_ID
----------
```

3.identify customer with maximum purchases

Purpose: To find the customer who has spent the highest total amount on purchases.

step 1 calculate customer-wise purchase

```
SQL> SELECT
  2      Customer_ID,
  3      SUM(Total_Amount) AS TOTAL_PURCHASE
  4  FROM Orders
  5  GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
          1          75000
          2          73000
          3          31000
          4          15500
          5           3500
```

step 2 find the maximum purchase

```
SQL> SELECT MAX(TOTAL_PURCHASE) AS MAX_PURCHASE
  2  FROM
  3  (
  4      SELECT Customer_ID,
  5             SUM(Total_Amount) AS TOTAL_PURCHASE
  6      FROM Orders
  7      GROUP BY Customer_ID
  8  );

MAX_PURCHASE
------------
       75000
```

3.1 display customer details with maximum purchase

Purpose: To display the details of the customer who made the highest total purchase.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name,
  4      c.Last_Name,
  5      SUM(o.Total_Amount) AS Total_Purchase
  6  FROM Customer c
  7  JOIN Orders o
  8      ON c.Customer_ID = o.Customer_ID
  9  GROUP BY
 10      c.Customer_ID,
 11      c.First_Name,
 12      c.Last_Name
 13  HAVING SUM(o.Total_Amount) =
 14  (
 15      SELECT MAX(Total_Purchase)
 16      FROM
 17      (
 18          SELECT SUM(Total_Amount) AS Total_Purchase
 19          FROM Orders
 20          GROUP BY Customer_ID
 21      )
 22  );

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_PURCHASE
-------------------------------------------------- --------------
          1 Arun
Kumar                                                       75000
```

4. complex business queries

4.1 customer who placed orders above average order value

Purpose: To identify customers who placed orders with an amount greater than the average order value.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name,
  4      c.Last_Name,
  5      o.Order_ID,
  6      o.Total_Amount
  7  FROM Customer c
  8  JOIN Orders o
  9  ON c.Customer_ID = o.Customer_ID
 10  WHERE o.Total_Amount >
 11  (
 12      SELECT AVG(Total_Amount)
 13      FROM Orders
 14  );

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          ORDER_ID TOTAL_AMOUNT
-------------------------------------------------- ---------- ------------
          1 Arun
Kumar                                                 201        55000

          2 Priya
Devi                                                  207        50000

          3 Rahul
Raj                                                   208        25000
```

5. find products whose sales are above average

Purpose: To identify products whose total sales amount is greater than the average sales amount per product.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(oi.Subtotal) AS TOTAL_SALES
  5  FROM Product p
  6  JOIN Order_Item oi
  7  ON p.Product_ID = oi.Product_ID
  8  GROUP BY p.Product_ID, p.Product_Name
  9  HAVING SUM(oi.Subtotal) >
 10  (
 11      SELECT AVG(PRODUCT_SALES)
 12      FROM
 13      (
 14          SELECT SUM(Subtotal) AS PRODUCT_SALES
 15          FROM Order_Item
 16          GROUP BY Product_ID
 17      )
 18  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
       101
Laptop
     100000

       102
Mobile
      40000

PRODUCT_ID
----------
```

6. customer purchase report

Purpose: To generate a report showing each customer's total orders, total purchase amount, and average order amount.

```
SQL> SELECT
  2      c.Customer_ID,
  3      c.First_Name,
  4      c.Last_Name,
  5      COUNT(o.Order_ID) AS TOTAL_ORDERS,
  6      SUM(o.Total_Amount) AS TOTAL_PURCHASE,
  7      ROUND(AVG(o.Total_Amount), 2) AS AVERAGE_PURCHASE
  8  FROM Customer c
  9  JOIN Orders o
 10  ON c.Customer_ID = o.Customer_ID
 11  GROUP BY
 12      c.Customer_ID,
 13      c.First_Name,
 14      c.Last_Name
 15  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
-------------------------------------------------- ------------ --------------
AVERAGE_PURCHASE
----------------
          1 Arun
Kumar                                                         2         75000
          37500

          2 Priya
Devi                                                          2         73000
          36500

CUSTOMER_ID FIRST_NAME
----------- --------------------------------------------------
LAST_NAME                                          TOTAL_ORDERS TOTAL_PURCHASE
-------------------------------------------------- ------------ --------------
```

7. advanced product sales report

Purpose: To display product details, total quantity sold, total sales amount, and average selling amount.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(oi.Quantity) AS TOTAL_QUANTITY_SOLD,
  5      SUM(oi.Subtotal) AS TOTAL_SALES,
  6      ROUND(AVG(oi.Subtotal), 2) AS AVERAGE_SALES
  7  FROM Product p
  8  JOIN Order_Item oi
  9  ON p.Product_ID = oi.Product_ID
 10  GROUP BY
 11      p.Product_ID,
 12      p.Product_Name
 13  ORDER BY TOTAL_SALES DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SALES
------------------- ----------- -------------
       101
Laptop
                  2      100000         50000

       102
Mobile
                  2       40000         20000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SALES
------------------- ----------- -------------
```

8. category-wise advanced sales report

Purpose: To analyze each product category based on total products sold, quantity, and sales amount.

```
SQL> SELECT
  2      c.Category_ID,
  3      c.Category_Name,
  4      COUNT(DISTINCT p.Product_ID) AS TOTAL_PRODUCTS,
  5      SUM(oi.Quantity) AS TOTAL_QUANTITY_SOLD,
  6      SUM(oi.Subtotal) AS TOTAL_SALES,
  7      ROUND(AVG(oi.Subtotal), 2) AS AVERAGE_SALES
  8  FROM Category c
  9  JOIN Product p
 10  ON c.Category_ID = p.Category_ID
 11  JOIN Order_Item oi
 12  ON p.Product_ID = oi.Product_ID
 13  GROUP BY
 14      c.Category_ID,
 15      c.Category_Name
 16  ORDER BY TOTAL_SALES DESC;

CATEGORY_ID CATEGORY_NAME                                      TOTAL_PRODUCTS
----------- -------------------------------------------------- --------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SALES
------------------- ----------- -------------
          1 Electronics                                                 3
                  6      170000      28333.33

          2 Clothing                                                    2
                 12       14000       4666.67

          5 Accessories                                                 1
                  2        6000          3000


CATEGORY_ID CATEGORY_NAME                                      TOTAL_PRODUCTS
----------- -------------------------------------------------- --------------
TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_SALES
------------------- ----------- -------------
```